Actividad 3: Lista enlazada simple: Búsqueda, modificación y eliminación

- **Algoritmo de búsqueda:**
  - ✓ **Criterio definido por el equipo:**
    Se buscará la solicitud utilizando su código como identificador único.
  - ✓ **Descripción:**
    El algoritmo recorre secuencialmente la lista desde el nodo inicio. Compara el código buscado con el código almacenado en cada nodo. Si lo encuentra, devuelve el nodo; si llega al final (null) sin éxito, indica que no existe.

➢ **Pseudocodigo:**

```
Algoritmo BuscarSolicitud
    Definir actual Como Nodo
    Definir codigoBuscado Como Cadena

    Escribir "Ingrese el codigo de la solicitud a buscar:"
    Leer codigoBuscado

    actual <- inicio
    Mientras actual <> NULL Hacer
        Si actual.codigo = codigoBuscado Entonces
            Escribir "Solicitud encontrada: ",
actual.descripcion, " - Estado: ", actual.estado
            Retornar actual
        FinSi
        actual <- actual.siguiente
    FinMientras

    Escribir "La solicitud no se encuentra registrada."
    Retornar NULL
FinAlgoritmo
```

- **Algoritmos para modificar un nodo encontrado:**

  - ✓ **Descripción:**
    Este algoritmo utiliza la lógica de búsqueda para ubicar una solicitud especifica. Una vez localizada, permite actualizar su información, por ejemplo, cambiar el estado de “pendiente” a

"atendido". Esta operación solo altera los datos internos del nodo, los enlaces de la estructura permanecen intactos.

➢ **Pseudocodigo:**

```
Algoritmo ModificarEstadoSolicitud
    Definir nodoEncontrado Como Nodo
    Definir nuevoEstado Como Cadena

    nodoEncontrado <- BuscarSolicitud() // Llama al
algoritmo anterior

    Si nodoEncontrado <> NULL Entonces
        Escribir "Ingrese el nuevo estado para la
solicitud:"
        Leer nuevoEstado
        nodoEncontrado.estado <- nuevoEstado
        Escribir "Estado actualizado correctamente."
    FinSi
FinAlgoritmo
```

- **Algoritmo para eliminar un nodo y reajuste de enlaces**

  ✓ **Descripción:**
  Para eliminar una solicitud, se debe localizar el nodo y desconectarlo de la lista. El reajuste de enlaces depende de su posición:
  1. **Si es el primer nodo:** La variable inicio pasa a apuntar al segundo nodo (inicio.siguiente).
  2. **Si esta en el medio o al final:** Se necesita un puntero anterior. El enlace siguiente del nodo anterior se modifica para que apunte al nodo que esta después del nodo a eliminar (actual.siguiente), saltándose así el nodo eliminado.

➢ **Pseudocodigo:**

```
Algoritmo EliminarSolicitud
    Definir actual, anterior Como Nodo
    Definir codigoEliminar Como Cadena

    Escribir "Ingrese el codigo de la solicitud a
eliminar:"
    Leer codigoEliminar
```

```
    Si inicio = NULL Entonces
        Escribir "La lista esta vacia."
        Retornar
    FinSi

    // Caso 1: El nodo a eliminar es el primero
    Si inicio.codigo = codigoEliminar Entonces
        inicio <- inicio.siguiente
        Escribir "Solicitud eliminada."
        Retornar
    FinSi

    // Caso 2: El nodo a eliminar esta en el medio o al
final
    actual <- inicio
    anterior <- NULL

    Mientras actual <> NULL Y actual.codigo <>
codigoEliminar Hacer
        anterior <- actual
        actual <- actual.siguiente
    FinMientras

    Si actual = NULL Entonces
        Escribir "Solicitud no encontrada."
    SiNo
        // Reajuste de enlaces: El anterior salta al nodo
que sigue del actual
        anterior.siguiente <- actual.siguiente
        Escribir "Solicitud eliminada."
    FinSi
FinAlgoritmo
```

- **Comparación de la eficiencia básica (Inserción vs. Eliminación según punto de intervención)**

  La eficiencia de ambas operaciones en una lista enlazada simple depende fundamentalmente de la posición en la que se interviene, ya que no se tiene acceso directo por índice:

  - ✓ **Intervención al inicio:**

- **Inserción:** Complejidad $O(1)$. Es altamente eficiente porque solo requiere crear el nodo, enlazarlo al inicio actual y actualizar el puntero inicio.
- **Eliminación:** Complejidad $O(1)$. Igualmente eficiente, solo requiere actualizar el puntero inicio para que apunte al segundo nodo.

✓ **Intervención al final:**

- **Inserción:** Complejidad $O(n)$. Como la lista simple (sin puntero adicional a la cola) solo conoce el inicio, debe recorrer todos los nodos uno por uno hasta encontrar el que apunta a null para enlazar el nuevo nodo.
- **Eliminación:** Complejidad $O(n)$. Requiere recorrer toda la estructura para llegar al final y además, mantener un rastro del penúltimo nodo para que su puntero cambie a null.

✓ **Intervención en el medio (después del nodo / buscando un nodo)**:

- **Inserción y Eliminación:** Ambas tienen complejidad $O(n)$ en el peor caso. El acto de reajustar los enlaces en si mismo toma $O(1)$, pero el proceso previo de recorrer la lista para encontrar el nodo especifico donde se hará la modificación consume un tiempo proporcional al numero de elementos.